

Experiment 6

Continuous Integration with Jenkins

What is a CI Pipeline?

A Continuous Integration (CI) Pipeline automates the process of building, testing, and integrating code changes every time code is committed to the repository. This pipeline:

- Automatically checks out the latest code.
- Compiles the application.
- Runs tests to catch errors early.
- Notifies the team of build/test results.

Why Use Jenkins for CI?

- **Automation:** Jenkins automates the build and test cycle, reducing manual intervention.
- **Immediate Feedback:** Developers get rapid notifications of any integration issues.
- **Extensibility:** With hundreds of plugins available, Jenkins can integrate with version control systems, build tools (Maven, Gradle), testing frameworks, and more.
- **Pipeline as Code:** Using Jenkins Pipelines (defined in a Jenkinsfile), you can manage the CI process as part of your source code repository.

Prerequisites

Before proceeding, ensure the following:

- ✓ **Jenkins is installed and running** on your system.
- ✓ **Git is installed** and configured in Jenkins. (Verify with `git --version`).
- ✓ **Maven is installed** and configured in Jenkins. (Check with `mvn -version`).
- ✓ **Selenium Maven Project is ready** with test cases (`src/test/java`).
- ✓ **Project is stored in two places:**
 - Locally on your system (e.g., `D:\Idea Projects\MVNGRDLDEMO`).
 - Pushed to **GitHub** with a valid repository link.
- ✓ **Jenkins has access to the GitHub repository** (via credentials).

Before doing experiment 6, we need to open command prompt and type
`java -jar jenkins.war`

1. Running a Selenium Java Test from a Local Maven Project

Step 1: Create a New Jenkins Job

1. Go to **Jenkins Dashboard** → Click **New Item**.
2. Enter a project name → Select **Freestyle Project**.
3. Click **OK**.

Step 2: Configure the Build Step

1. Scroll to **Build** → Click **Add build step** → **Execute Windows Batch Command**.
2. Enter the following commands (**ensure correct navigation to project directory**):

```
cd D:\Idea Projects\MVNGRDLDEMO  
mvn test
```
3. Click **Save** → Click **Build Now** to execute the test.

2. Running Selenium Tests from a GitHub Repository via Jenkins

Step 1: Set Up a New Jenkins Job for GitHub Project

1. Go to **Jenkins Dashboard** → Click **New Item**.
2. Enter a project name → Select **Freestyle Project**.
3. Click **OK**.

Step 2: Configure Git Repository in Jenkins

1. Under **Source Code Management**, select **Git**.
2. Enter your GitHub repository URL (e.g., <https://github.com/your-repo-name.git>).
3. Select the **Git credentials** configured earlier.

Step 3: Add Build Step for Maven

1. Scroll to **Build** → Click **Add build step** → **Execute Windows Batch Command**.
2. Enter the Maven test command:

```
mvn test
```
3. Click **Save**.

Step 4: Trigger the Build

1. Click **Build Now** to fetch the code from GitHub and execute the Selenium tests.
2. Check the **Console Output** to verify test execution.